

Greedy snap

Global placement leaves fractional coordinates, the float seed puts A near (3.7, 1.2) and B near (4.1, 1.4)

The idea

- Snap quantizes coordinates but does not remove overlap
- After snap, A and B still fail with overlap A/B
- Teaching point
- <!-- algorithm-walkthrough -->

Float placement from global place

GREEDY SNAP

Float placement from global place

Step 1 / 5 · float-seed · module02-01-greedy-snap

chip 12x6 · 3 rows · site 1



Explanation

After global placement, cells sit at fractional coordinates —A near (3.7, 1.2), B near (4.1, 1.4). Nothing is site-aligned yet; that is the float seed for greedy snap.

- Fractional x and y
- Middle row target for A–C
- D, E, F also float

```
starter: FLOAT_PLACEMENT  
legal before snap: false
```

Float placement from global place

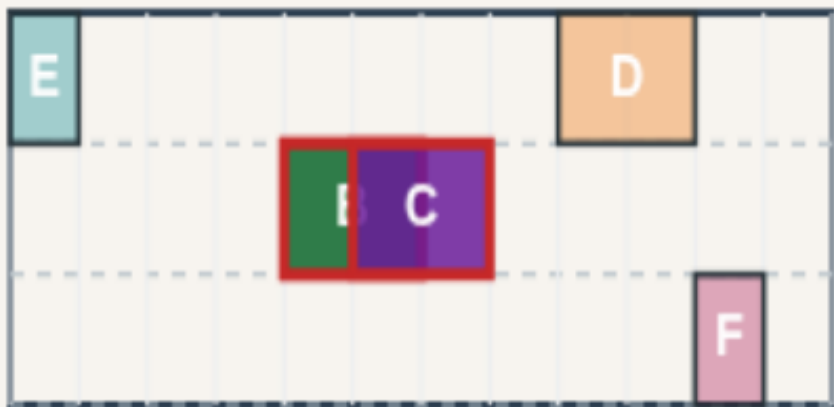
Snap every cell to nearest site

GREEDY SNAP

Snap every cell to nearest site

Step 2 / 5 · snap-all · module02-01-greedy-snap

chip 12×6 · 3 rows · site 1



Explanation

Greedy snap rounds x to the nearest site and y to the nearest row. A lands at (4, 2)—same as B. C snaps to (5, 2). D, E, and F snap to their row neighbors.

- snapX with cell width clamp
- snapY to nearest ROW_YS
- No overlap repair yet

A → (4, 2)
B → (4, 2)
C → (5, 2)

Snap every cell to nearest site

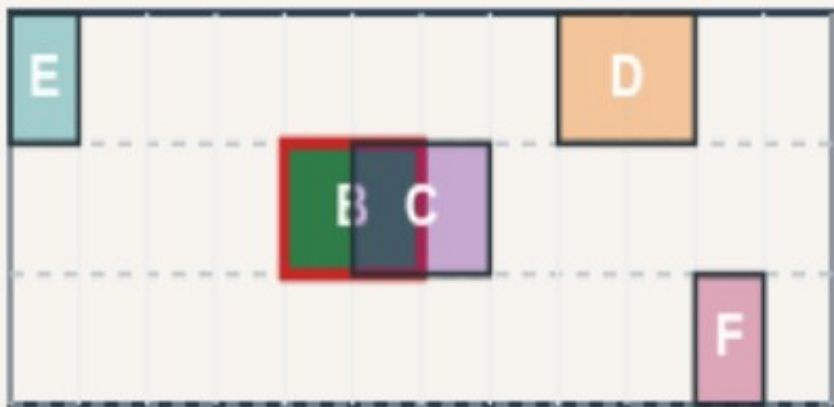
Still illegal: A/B overlap

GREEDY SNAP

Still illegal: A/B overlap

Step 3 / 5 · still-overlap · module02-01-greedy-snap

chip 12x6 · 3 rows · site 1



Explanation

After snap, A and B still share (4, 2). The legality report again says overlap A/B. Snap alone does not legalize—it only quantizes coordinates.

- floatSnapLegal: false
- Same failure as overlap seed
- C nearby but not stacked

```
legal: false
reason: overlap A/B
A,B @ (4,2)
```

Still illegal: A/B overlap

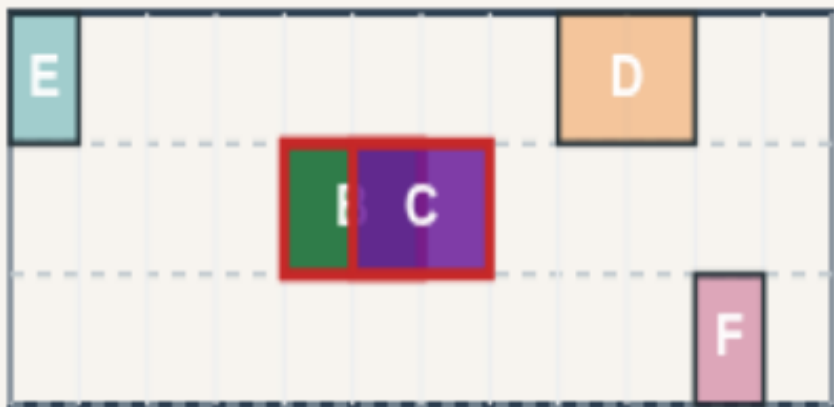
Teaching point: snap $\neq$ legal

GREEDY SNAP

Teaching point: snap $\neq$ legal

Step 4 / 5 · snap-not-legal · module02-01-greedy-snap

chip 12×6 · 3 rows · site 1



Explanation

Students often assume rounding fixes everything. On this instance greedy snap creates the same middle-row pile as the integer overlap seed. Legalization needs a second phase.

- Snap is necessary, not sufficient
- Check legality after snap
- Next: overlap removal or Abacus

greedy snap only
overlap remains

Teaching point: snap $\neq$ legal

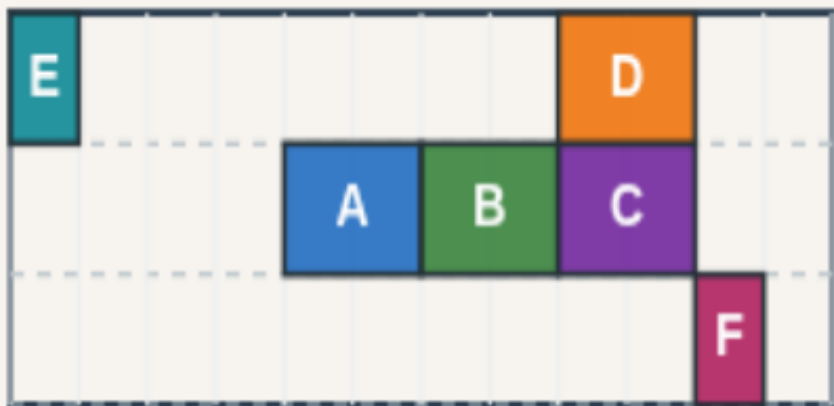
Next: overlap removal / Abacus

GREEDY SNAP

Next: overlap removal / Abacus

Step 5 / 5 · next-phase · module02-01-greedy-snap

chip 12×6 · 3 rows · site 1



Explanation

Follow snap with per-row packing or Abacus row assignment. Overlap removal spreads A, B, C along row two; Abacus spreads them across rows with lower displacement.

- overlap-removal: disp 6
- abacus: disp 4
- Both start from overlap or float snap

```
overlapRemovalDisp: 6  
abacusDisp: 4
```

Next: overlap removal / Abacus

Browser lab track

- In the browser lab track, open the **greedy-snap** lab from the tools shelf
- Load the overlap or float starter, run the legalizer once
- Work the challenges that lock the goldens

Implement track

- In the implement track, open this module's examples and the course `common/` solvers
- Parse `tiny_legal.json`, run the algorithm with deterministic coordinates
- Match the browser goldens before you claim the checklist

Pitfalls

- Common traps

Your turn

- Complete the checklist for at least one track, preferably both
- Implement until your metrics match the starter goldens
- When you're ready, take the short quiz, then continue to the next module